

MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work n. 2
of Operating Systems

Meet the OS I will build

Supervisers:

Dumitrașcu Marius, *University assistant*

Reițman Patricia, *University assistant*

DISA, FCIM, UTM

Chișinău – 2026

Table of Contents

1 Introduction **3**

2 Before we start **4**

3 Building the OS **8**

3.1 Build it and boot it 8

3.2 Use xv6 as the Unix it is 8

3.2.1 Observations 12

3.3 Read the source 12

3.3.1 Observations 17

3.4 Write first xv6 program 17

4 CONCLUSIONS **22**

1 Introduction

The goal of this laboratory work is to become familiar with xv6, a small Unix-like teaching operating system. xv6 is a modern reimplementation of Unix Version 6, containing the main components of a real operating system while remaining small enough for its source code to be studied directly.

The operating system is built from source and executed on a RISC-V machine provided by an emulator. This makes it possible to use xv6 as a small Unix system, inspect both its user programs and kernel code, and run a program written specifically for it.

2 Before we start

The work is performed in an Ubuntu virtual machine running in VirtualBox. The Ubuntu system itself runs on the ARM64 architecture, while xv6 is built for RISC-V.

RISC-V is an open instruction set architecture developed at the University of California, Berkeley. Its development began in 2010 as a new architecture for computer architecture research and processor design. The name refers to the fifth generation of the Berkeley RISC architecture lineage.

An instruction set architecture defines the machine instructions understood by a processor together with fundamental aspects of its programming model. RISC-V can therefore be implemented by physical processors as well as by software emulators. xv6 uses RISC-V as its target architecture, independently of the architecture on which the development tools themselves are running.

Since Ubuntu is running on ARM64 while the generated xv6 code must contain RISC-V instructions, a RISC-V cross-compilation toolchain is used. The `riscv64-linux-gnu-gcc` compiler translates C source files into object files containing 64-bit RISC-V machine code.

The separate object files are then combined by `riscv64-linux-gnu-ld`. The linker resolves references between them and produces the binaries that form the kernel and user programs. The linker, assembler, `objdump`, `objcopy`, and several related tools are provided by GNU Binutils for RISC-V.

The build sequence is controlled by GNU Make according to the rules stored in the xv6 `Makefile`. It invokes the required compilation and linking commands and provides targets for building and starting the system.

The resulting RISC-V machine code cannot be executed directly by the ARM64 processor used by Ubuntu. QEMU provides the required RISC-V system in software. The `qemu-system-riscv64` emulator represents a complete 64-bit RISC-V computer on which xv6 can boot and execute.

The execution environment therefore consists of several layers:

Physical computer $\rightarrow$ VirtualBox $\rightarrow$ Ubuntu ARM64 $\rightarrow$ QEMU RISC-V $\rightarrow$ xv6.

GDB Multiarch is also part of the development toolchain. It can debug programs for architectures other than that of the host machine and can be connected to an emulated RISC-V system to inspect execution, memory, processor registers, and kernel state.

The xv6 sources are expected to be stored in `~/xv6-riscv`. To begin, we will look for this directory in the home folder.

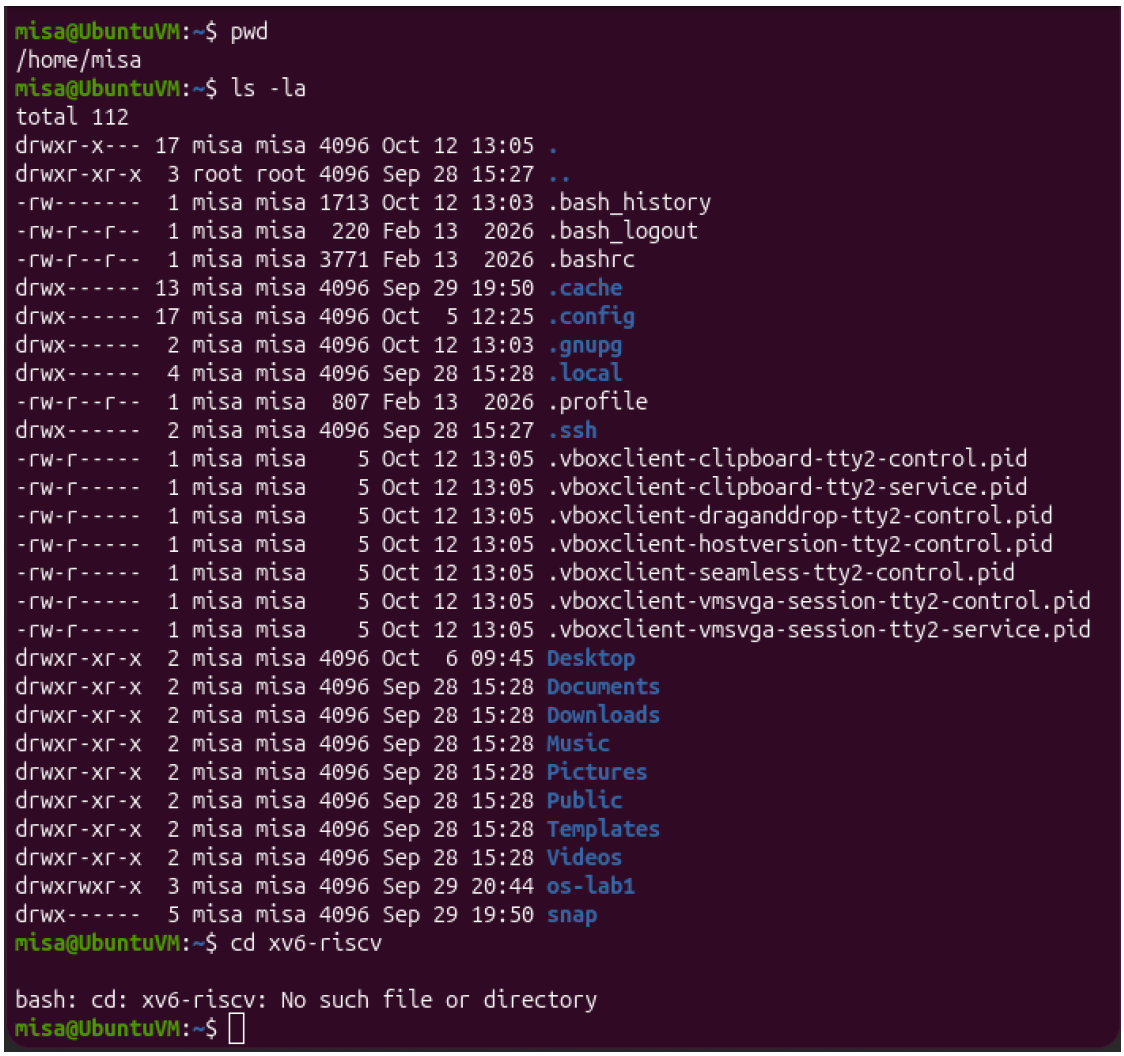


Figure 2.1 - Checking the home directory for the xv6 source code

The `xv6-riscv` directory is not present in the home folder, so the xv6 source code still needs to be obtained.

Before obtaining the source code, we check whether the tools required to build and run xv6 are available.

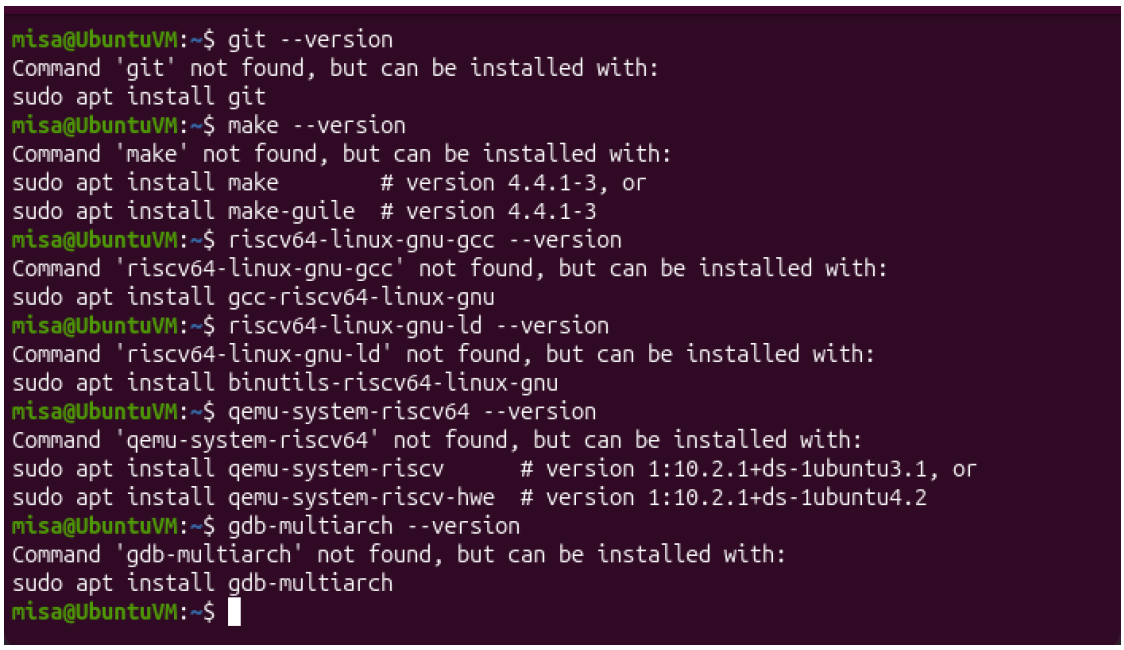


Figure 2.2 - Checking the availability of the required development tools

None of the required tools are currently available in the system. The development environment therefore needs to be prepared before xv6 can be built and executed.

The missing tools are installed through the Ubuntu package manager.

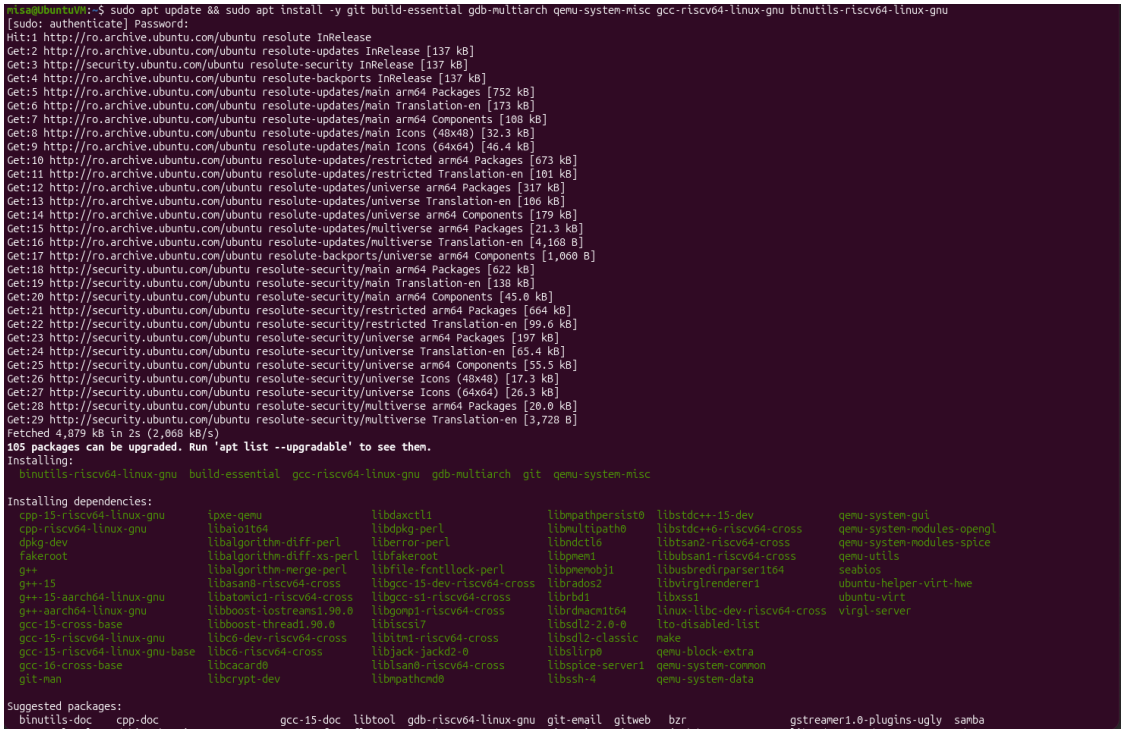


Figure 2.3 - Installing the xv6 development toolchain

The installed tools were then checked again.

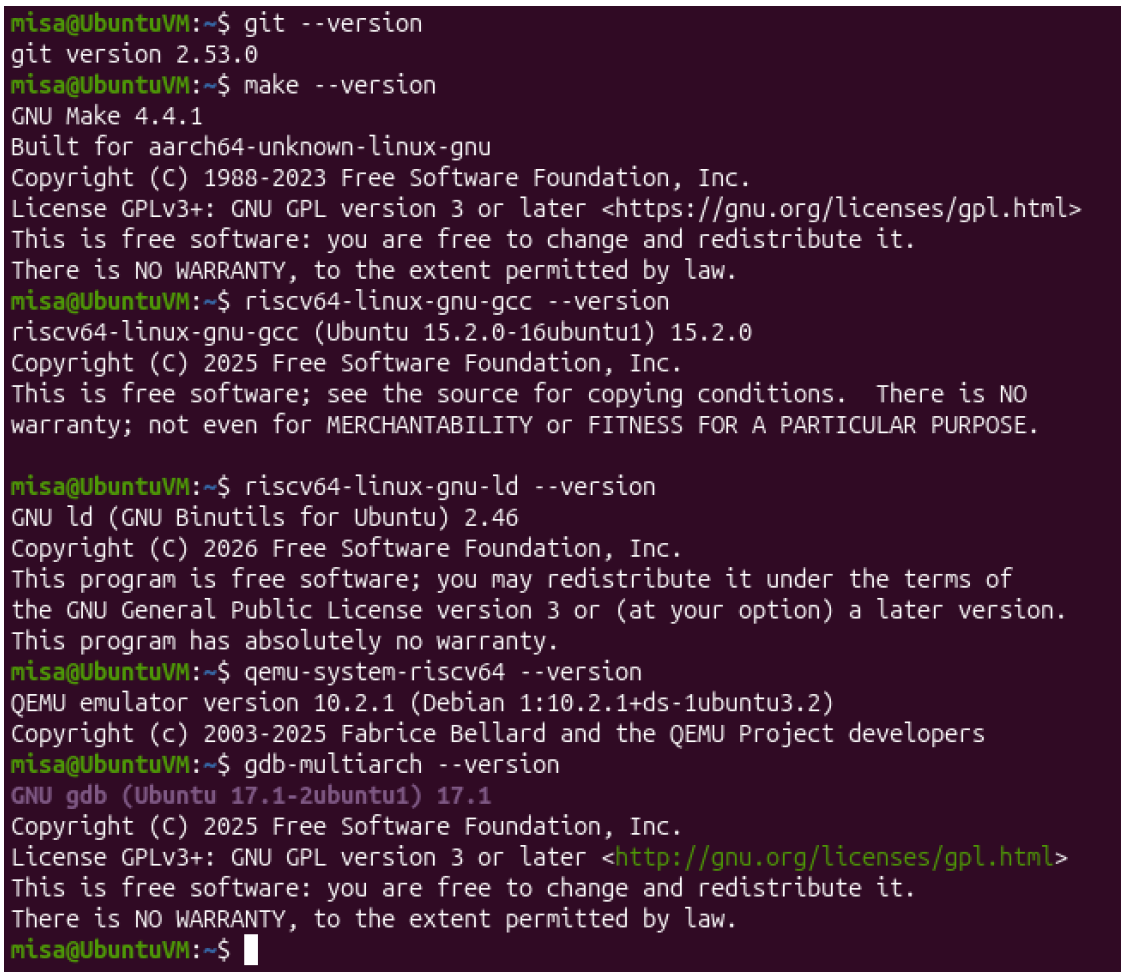


Figure 2.4 - Verification of the installed development tools

All required development tools are now installed and available in the system. The environment is ready for working with xv6.

With the development environment ready, the xv6 source code can now be obtained from the official repository.



Figure 2.5 - Cloning and inspecting the xv6 source repository

The xv6 source tree is now available in `~/xv6-riscv`. It contains the kernel sources, user programs,

the Makefile, and the other files required to build the system.

3 Building the OS

3.1 Build it and boot it

The xv6 kernel and user programs were built from source using the RISC-V toolchain. Make compiled and linked the required components, created the filesystem image, and launched the resulting system in QEMU.

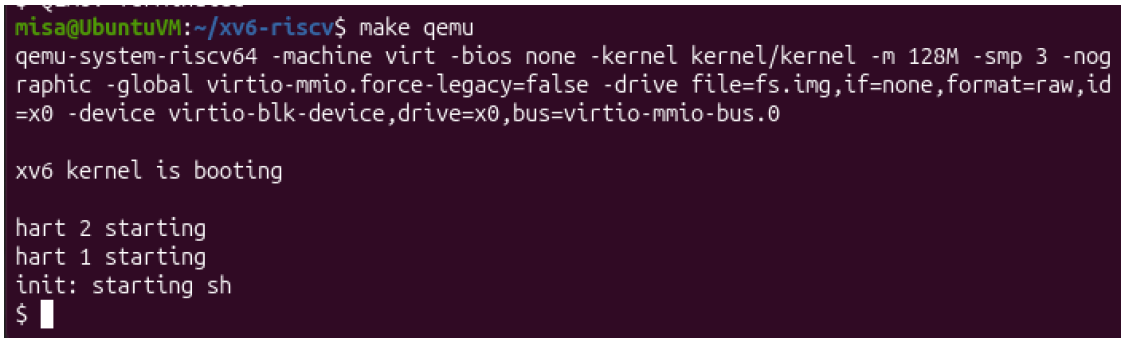


Figure 3.1 - Booting xv6 in the QEMU RISC-V emulator

The boot completed successfully. After initializing the system, xv6 started its shell and presented the command prompt.

3.2 Use xv6 as the Unix it is

After booting, xv6 provides a simple Unix shell through which user programs can be executed. Its command interface resembles that of Linux, although the number of available programs and supported features is considerably smaller.

We begin by examining the contents of the xv6 filesystem using `ls`. The command lists files and directories together with their types, inode numbers, and sizes in bytes. The output includes familiar Unix utilities such as `cat`, `echo`, `grep`, `ls`, and `wc`, as well as several programs for testing the operating system.

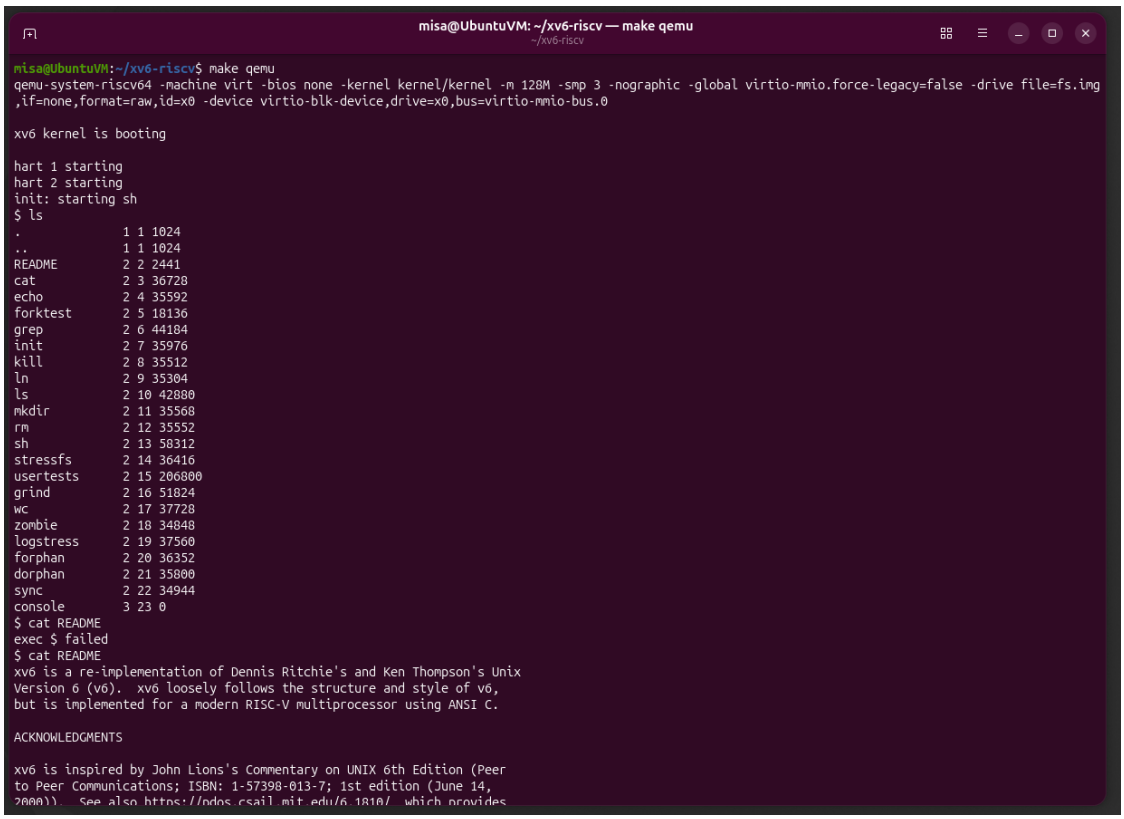


Figure 3.2 - Exploring the xv6 filesystem using ls

The `README` file is then examined using `cat`, which reads a file and writes its contents to standard output. The file describes the origins of xv6, acknowledges its contributors, and provides information about building and running the system.

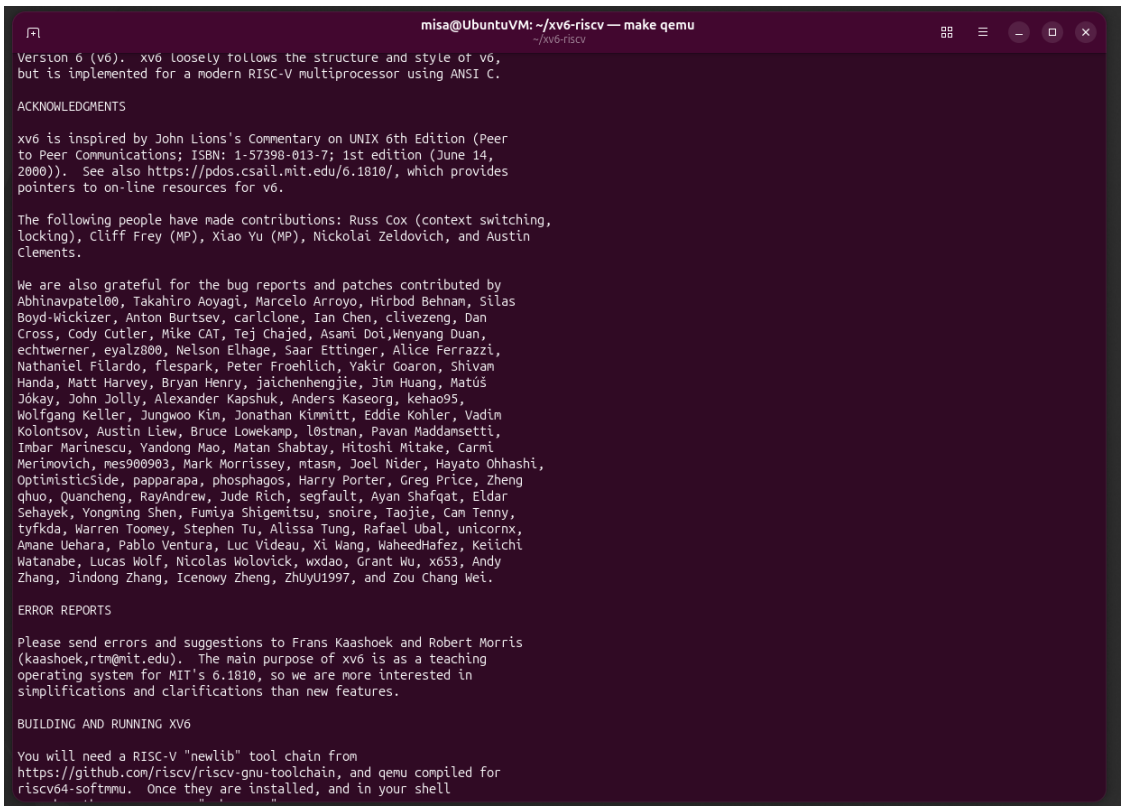


Figure 3.3 - Reading the xv6 README file

The `echo` command demonstrates basic terminal output by printing the supplied arguments. Executing it with the words `hello xv6` produces the corresponding text in the terminal.

Next, a pipeline is tested using `ls | grep c`. The `ls` command generates a directory listing, while `grep` searches its input for lines containing a specified pattern. Here, the pattern is the character `c`.

The pipe operator `|` connects the standard output of `ls` to the standard input of `grep`. As a result, only entries containing the character `c` are displayed, including `cat`, `echo`, `wc`, `sync`, and `console`.

This demonstrates two important operating system mechanisms: process management and interprocess communication. The shell creates processes for the connected programs, while the kernel provides a pipe through which data can be transferred between them.

The `wc` utility is then used to examine the `README` file. The command counts lines, words, and bytes. The result shows that the file contains 48 lines, 336 words, and 2441 bytes.

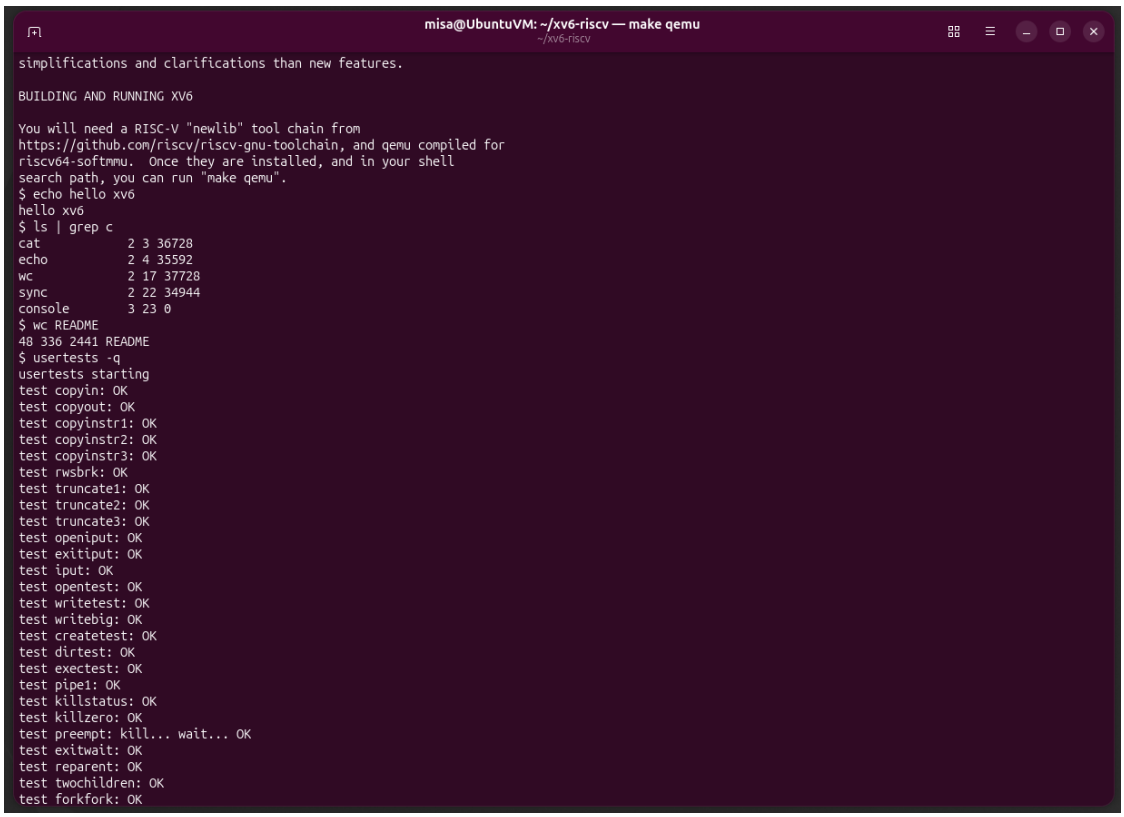


Figure 3.4 - Testing shell commands, pipelines, and file statistics

Finally, the operating system is tested using `usertests -q`. The `usertests` program runs a collection of tests covering various kernel services, including process management, memory allocation, filesystem operations, and system calls.

The `-q` option enables quiet mode, suppressing some of the detailed progress messages produced by the test suite. Individual test results are still displayed.

During execution, several tests deliberately trigger exceptions to verify how the kernel handles invalid memory accesses and other exceptional conditions. These produce diagnostic messages such as `usertrap(): unexpected scause`, followed by the corresponding test results.

The test suite completes with the message `ALL TESTS PASSED`, confirming that all executed tests finished successfully.

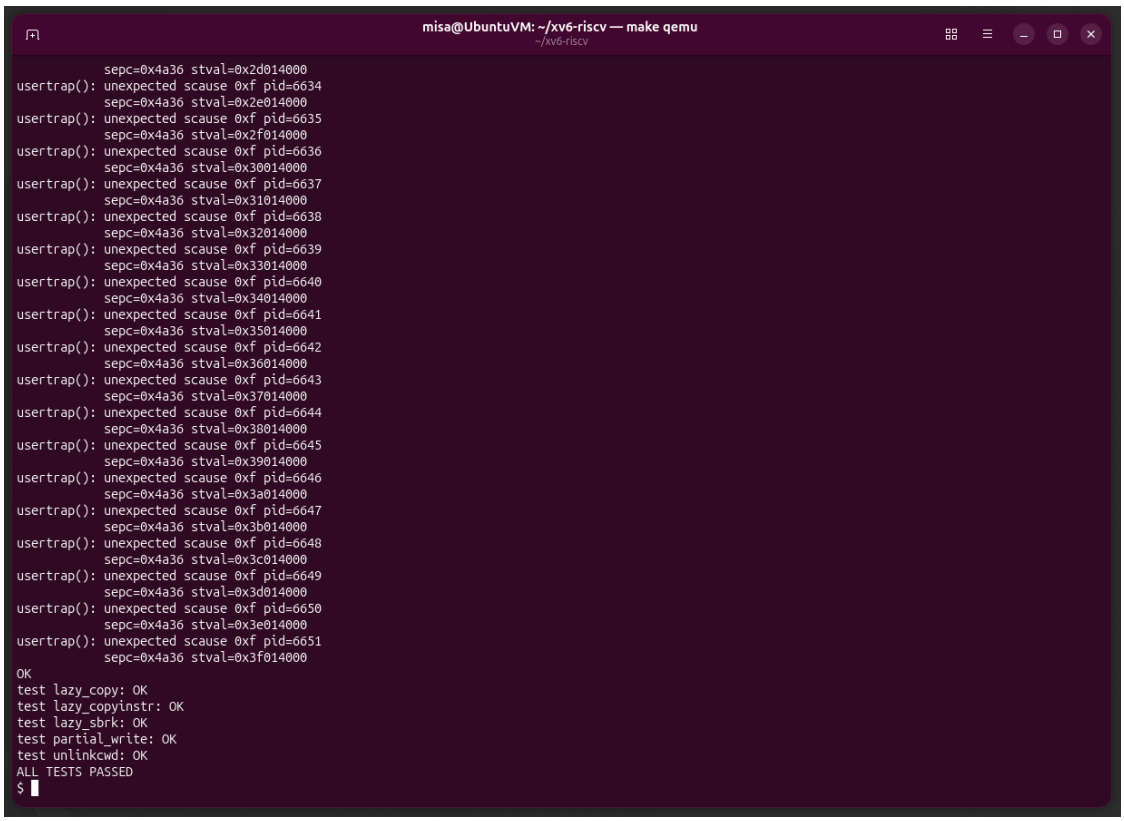


Figure 3.5 - Successful completion of the xv6 user test suite

The experiments demonstrate that xv6 supports the fundamental Unix mechanisms needed to execute user programs, access files, communicate between processes, and interact through a shell. Its interface resembles a Linux shell, but provides a smaller collection of commands and features.

3.2.1 Observations

1. **List three programs xv6 ships with.**

   xv6 includes several standard Unix utilities, including `cat`, `echo`, and `grep`. These programs are available directly from the xv6 shell.

2. **You just used a pipe (|) inside xv6. Which two OS features must exist for a pipe to work?**

   A pipeline requires process management and inter-process communication (IPC). The operating system must support the execution of separate processes and provide a pipe through which one process can send data to another.

3. **In one sentence: how does the xv6 shell compare to the Linux shell from Lab 1?**

   The xv6 shell supports familiar Unix commands and pipelines, but provides fewer utilities and features than a modern Linux shell.

3.3 Read the source

The xv6 source tree is organized around two main directories: `kernel/` and `user/`. The first contains the operating system kernel, including process scheduling, memory management, filesystem opera-

tions, and system call handlers. The second contains user-space programs that rely on the kernel to access operating system services.

To examine this separation, the source code of two familiar utilities, `ls` and `cat`, was inspected.

The `sed` utility allows selected portions of a text file to be displayed. The option `-n` suppresses automatic printing, while the expression `1,40p` prints lines 1 through 40. This makes it possible to inspect a specific part of a source file without displaying its entire contents.

The first 40 lines of `user/ls.c` contain the header inclusions, the `fmtname()` function, and the beginning of `ls()`.

The `fmtname()` function extracts the final component of a file path and prepares it for display. The `ls()` function begins by opening the specified path using `open()` and obtaining file information through `fstat()`.

```
misa@UbuntuVM:~/xv6-riscv$ sed -n "1,40p" user/ls.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

char *
fmtname(char *path)
{
  static char buf[DIRSIZ + 1];
  char *p;

  // Find first character after last slash.
  for (p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;

  // Return blank-padded name.
  if (strlen(p) >= DIRSIZ)
    return p;
  memmove(buf, p, strlen(p));
  memset(buf + strlen(p), ' ', DIRSIZ - strlen(p));
  buf[sizeof(buf) - 1] = '\0';
  return buf;
}

void
ls(char *path)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if ((fd = open(path, O_RDONLY)) < 0) {
    fprintf(2, "ls: cannot open %s\n", path);
    return;
  }

  if (fstat(fd, &st) < 0) {
```

Figure 3.6 - Examining the source code of the ls utility

The next program examined is `user/cat.c`. Its first 30 lines contain a global buffer of 512 bytes and the `cat()` function responsible for transferring data from an input file descriptor to standard output.

The program repeatedly calls `read()` to obtain data and `write()` to send it to the terminal. The loop continues until `read()` returns zero, indicating the end of the input.

The value returned by `read()` determines how many bytes are written. A negative return value

indicates an error, while a failed or incomplete write produces an error message and terminates the program.

The file descriptor passed to `cat()` identifies the input source. The value `1` used in `write()` represents standard output, while `2` used in `fprintf()` represents standard error.

```
misa@UbuntuVM:~/xv6-riscv$ sed -n "1,30p" user/cat.c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}

int
main(int argc, char *argv[])
{
  int fd, i;

  if (argc <= 1) {
    cat(0);
```

Figure 3.7 - Examining the source code of the cat utility

The functions `read()` and `write()` provide an interface between user programs and the operating system kernel. Their declarations can be found in `user/user.h`, together with other system calls such as `fork()`, `exit()`, `wait()`, `pipe()`, and `open()`.

The header also declares ordinary user-space functions, including string manipulation, memory allocation, and formatted output routines.

```
misa@UbuntuVM:~/xv6-riscv$ cat user/user.h
#define SBRK_ERROR ((char *)-1)

struct stat;

// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
int mknod(const char *, short, short);
int unlink(const char *);
int fstat(int fd, struct stat *);
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);

// ulib.c
int stat(const char *, struct stat *);
char *strcpy(char *, const char *);
void *memmove(void *, const void *, int);
char *strchr(const char *, char c);
int strcmp(const char *, const char *);
char *gets(char *, int max);
uint strlen(const char *);
void *memset(void *, int, uint);
int atoi(const char *);
int memcmp(const void *, const void *, uint);
void *memcpy(void *, const void *, uint);
char *sbrk(int);
char *sbrklazy(int);

// printf.c
void fprintf(int, const char *, ...) __attribute__((format(printf, 2, 3)));
void printf(const char *, ...) __attribute__((format(printf, 1, 2)));

// umalloc.c
void *malloc(uint);
void free(void *);
```

Figure 3.8 - System call declarations in user/user.h

The corresponding kernel-side implementations of the file-related system calls are located in `kernel/sysf`

The `grep` command was used to locate the `sys_read` and `sys_write` functions. The option `-n` includes line numbers in the output, while the pattern `sys_read\|sys_write` matches either function name.

The output is passed through `head`, which displays the first ten lines by default.

The results locate `sys_read()` at line 69 and `sys_write()` at line 83 of `kernel/sysfile.c`.

```
misa@UbuntuVM:~/xv6-riscv$ grep -n "sys_read\|sys_write" kernel/sysfile.c | head
69:sys_read(void)
83:sys_write(void)
```

Figure 3.9 - Locating read and write system call handlers in the kernel

The source code demonstrates how user-space programs access operating system services through system calls. The user program requests an operation, while the kernel performs it using its privileged access to system resources.

3.3.1 Observations

1. **Which system calls does user/cat.c use, and what does each one ask the kernel to do?**

   The examined `cat()` function uses `read()` to obtain bytes from a file descriptor and `write()` to send those bytes to standard output. It also uses `exit()` to terminate the process when an error occurs.

   The complete program additionally uses file-related system calls such as `open()` and `close()` to access and release input files.

2. **In which kernel file, and on which line, is sys_read implemented?**

   The `sys_read()` function is implemented in `kernel/sysfile.c`, beginning at line 69.

3. **In one sentence: what is the difference between the code in kernel/ and the code in user/?**

   The `kernel/` directory contains privileged operating system code responsible for managing system resources, while `user/` contains ordinary programs that access these resources through system calls.

3.4 Write first xv6 program

The next task is to create a user program called `sleep`, which suspends its execution for a specified number of clock ticks.

The program is created in `user/sleep.c`. It receives command-line arguments through `argc` and `argv`. The first argument specifies the number of ticks, which is converted from a string to an integer using `atoi()`.

The program checks that exactly one argument has been provided. If the argument is missing or the number of arguments is incorrect, `fprintf()` prints a usage message to standard error (file descriptor 2), and `exit(1)` terminates the program. Otherwise, the requested delay is performed, followed by `exit(0)` to indicate successful completion.

The initial implementation followed the provided example and used the `sleep()` system call. The source file was created and edited using the Ubuntu terminal.

```
misa@UbuntuVM:~/xv6-riscv$ cd user
misa@UbuntuVM:~/xv6-riscv/user$ touch sleep.c
misa@UbuntuVM:~/xv6-riscv/user$ vim sleep.c
Command 'vim' not found, but can be installed with:
sudo apt install vim        # version 2:9.1.2141-1ubuntu4.6, or
sudo apt install vim-gtk3   # version 2:9.1.2141-1ubuntu4.6
sudo apt install vim-motif  # version 2:9.1.2141-1ubuntu4.6
sudo apt install vim-nox    # version 2:9.1.2141-1ubuntu4.6
sudo apt install neovim     # version 0.11.6-1
misa@UbuntuVM:~/xv6-riscv/user$ nano sleep.c
misa@UbuntuVM:~/xv6-riscv/user$ cat sleep.c
#include "kernel/types.h"
#include "user/user.h"
int
main(int argc, char *argv[])
{
 if(argc != 2){
 fprintf(2, "usage: sleep <ticks>\n");
 exit(1);
 }
 sleep(atoi(argv[1])); // system call into the kernel
 exit(0);
}
misa@UbuntuVM:~/xv6-riscv/user$ cd ..
misa@UbuntuVM:~/xv6-riscv$ nano Makefile
misa@UbuntuVM:~/xv6-riscv$ cat Makefile
K=kernel
U=user

OBJS = \
  $K/entry.o \
  $K/start.o \
  $K/console.o \
  $K/printk.o \
  $K/uart.o \
  $K/kalloc.o \
  $K/spinlock.o \
  $K/string.o \
  $K/main.o \
```

Figure 3.10 - Creating the sleep program and inspecting its initial implementation

To include the new program in the xv6 filesystem, the `UPROGS` variable in the `Makefile` was modified by adding `$U/_sleep`. This allows Make to compile and link the program and include its executable in the filesystem image.

```
UPROGS=\
        $U/_cat\
        $U/_echo\
        $U/_sleep\
        $U/_forktest\
        $U/_grep\
        $U/_init\
        $U/_kill\
        $U/_ln\
        $U/_ls\
        $U/_mkdir\
        $U/_rm\
        $U/_sh\
        $U/_stressfs\
        $U/_usertests\
        $U/_grind\
        $U/_wc\
        $U/_zombie\
        $U/_logstress\
        $U/_forphan\
        $U/_dorphan\
        $U/_sync\
```

During the first compilation attempt, GCC reported an implicit declaration of the `sleep()` function. The current xv6 source tree exposes the corresponding functionality through `pause(int)`, as declared in `user/user.h`.

The implementation was corrected by replacing the call to `sleep()` with `pause()`, while preserving the program name and its command-line interface.

The `pause()` system call suspends the calling process for the specified number of clock ticks. The argument is passed through `atoi(argv[1])`, allowing the user to specify the delay directly from the shell.

```
misa@UbuntuVM:~/xv6-riscv$ make qemu
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/misa/xv6-riscv=. -march
=rv64gc -std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin
-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-bu
iltin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -I. -fno-stack-protector -fno
-pie -no-pie   -c -o user/sleep.o user/sleep.c
user/sleep.c: In function ‘main’:
user/sleep.c:10:2: error: implicit declaration of function ‘sleep’ [-Wimplicit-function-declaration]
   10 |  sleep(atoi(argv[1])); // system call into the kernel
      |  ^~~~~
At top level:
cc1: note: unrecognized command-line option ‘-Wno-unknown-attributes’ may have been intended to silence earlier diagnostics
make: *** [<builtin>: user/sleep.o] Error 1
misa@UbuntuVM:~/xv6-riscv$ nano user/sleep.c
misa@UbuntuVM:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"
int
main(int argc, char *argv[])
{
 if(argc != 2){
 fprintf(2, "usage: sleep <ticks>\n");
 exit(1);
 }
 pause(atoi(argv[1])); // system call into the kernel
 exit(0);
}
misa@UbuntuVM:~/xv6-riscv$ nano Makefile
misa@UbuntuVM:~/xv6-riscv$ echo "Switch from spaces to tab in that list"
Switch from spaces to tab in that list
```

Figure 3.12 - Compilation error and correction of the sleep system call

After correcting the source code and adjusting the Makefile, the operating system was rebuilt using `make qemu`.

The build process compiled `user/sleep.c` into `user/sleep.o`, linked the resulting object file into the executable `user/_sleep`, and regenerated the filesystem image containing the new program.

The operating system then booted successfully in QEMU.

The program was tested with two commands. Executing `sleep 10` suspended the process for ten clock ticks before returning control to the shell.

Executing `sleep` without an argument triggered the argument validation and produced the expected message: `usage: sleep <ticks>`.

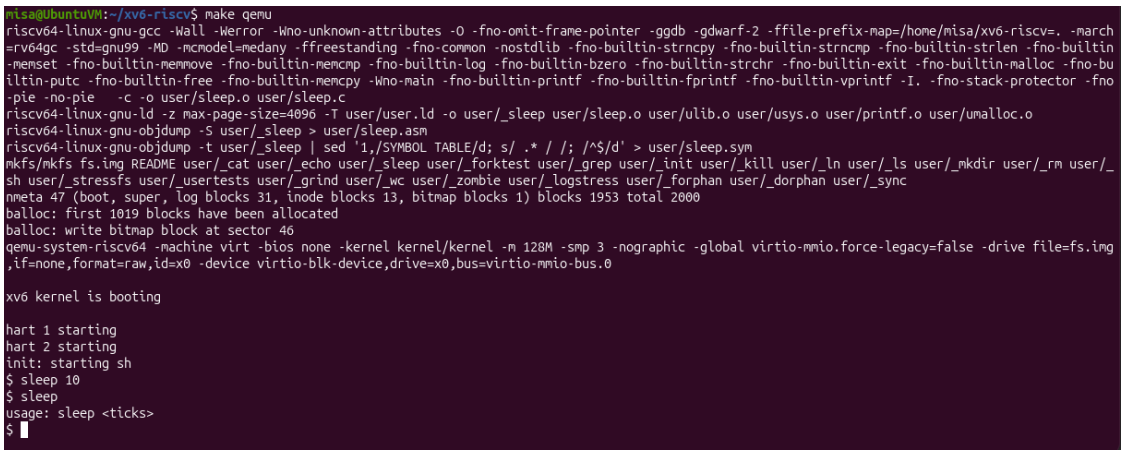


Figure 3.13 - Successful compilation and execution of the sleep program

Both tests produced the expected results. The new user program was successfully integrated into xv6 and executed using an existing kernel system call.

4 CONCLUSIONS

During this laboratory work, the xv6 operating system was successfully built from source and executed using the QEMU RISC-V emulator. The development environment was prepared by installing the required cross-compilation tools, and the xv6 source code was obtained from the official repository.

Working with the xv6 shell demonstrated the basic functionality of a Unix-like operating system, including file operations, process execution, and inter-process communication through pipes. The system also passed the complete user test suite, confirming the successful execution of its kernel functionality tests.

Examining the source code provided a clearer understanding of the separation between user-space programs and the operating system kernel. The implementations of `ls` and `cat` demonstrated how ordinary programs interact with system resources through system calls such as `read()` and `write()`.

Finally, a new user program, `sleep`, was implemented and integrated into the xv6 build process. An incompatibility between the provided example and the current xv6 version was identified and resolved by using `pause()` instead of `sleep()`. After rebuilding the system, the program executed successfully and handled both valid and missing command-line arguments.

Overall, the work provided practical experience with building, running, examining, and extending an operating system. Working directly with both user programs and kernel interfaces helped establish a better understanding of how operating system components cooperate to execute programs and provide services.